

Computability - Recitation 12

June 22th, 2026

1 The Time Hierarchy Theorem

To prove that giving a Turing Machine more time strictly increases the class of languages it can decide, we need a way for the machine to keep track of its own time limit without spending too much time just doing the counting. This brings us to the concept of time constructibility.

Intuitive Definition: A function $t(n)$ is time-constructible if a Turing Machine can act as its own “alarm clock,” successfully counting up to $t(n)$ steps without exceeding an $O(t(n))$ time limit to perform the count.

Definition 1.1. A function $t : \mathbb{N} \rightarrow \mathbb{N}$, where $t(n) = \Omega(n \log n)$, is called **time-constructible**, if the function that maps 1^n (n in unary form) to the binary representation of $t(n)$ is computable in time $O(t(n))$.

All commonly occurring functions that are $\Omega(n \log n)$ are time-constructible, including the functions $n \log n$, $n\sqrt{n}$, n^2 , and 2^n . All non-linear polynomials with integer coefficients and non-negative leading coefficient are time-constructible, and so are exponential functions.

We have the following time hierarchy theorem, which we state here without a proof:

Theorem 1.2. Let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a time-constructible function, then there exists a language L that is decidable in $O(t(n))$ time, but not decidable in $o\left(\frac{t(n)}{\log t(n)}\right)$ time.

Corollary 1.3. For any two real numbers $1 \leq \epsilon_1 < \epsilon_2$, we have $\text{TIME}(n^{\epsilon_1}) \subsetneq \text{TIME}(n^{\epsilon_2})$.

Corollary 1.4. $P \subsetneq \text{EXPTIME}$.

Proof. For every k , it holds that $n^k = O(2^n)$, so $\text{TIME}(n^k) \subseteq \text{TIME}(2^n)$, and therefore $P \subseteq \text{TIME}(2^n)$. By Theorem 1.1, we know that $\text{TIME}(2^n) \subsetneq \text{TIME}(4^n) \subseteq \text{EXPTIME}$. Thus, $P \subsetneq \text{EXPTIME}$. $\square$

Claim 1.5.

$$\exists k \in \mathbb{N} \text{ such that: } \bigcup_{i=1}^k \text{TIME}(n^i) = P$$

Proof. The claim is incorrect.

We assume for contradiction that there exists $k \in \mathbb{N}$ such that

$$P = \bigcup_{i=1}^k \text{TIME}(n^i).$$

For every $i \in [k]$,

$$\text{TIME}(n^i) \subseteq \text{TIME}(n^k),$$

so

$$P = \bigcup_{i=1}^k \text{TIME}(n^i) \subseteq \text{TIME}(n^k).$$

Notice that

$$n^k = o\left(\frac{n^{k+1}}{\log(n^{k+1})}\right)$$

and n^{k+1} is a time-constructible function. Since both conditions are satisfied, we can use the Time Hierarchy Theorem to show that

$$\text{TIME}(n^k) \subsetneq \text{TIME}(n^{k+1}).$$

Overall, we get that

$$P \subsetneq \text{TIME}(n^{k+1}),$$

in contradiction to the fact that

$$\text{TIME}(n^{k+1}) \subseteq P$$

(by the definition of P). □

2 The Space Hierarchy Theorem

While with respect to time complexity simulation comes with a logarithmic overhead, the same is not true for space and so the analogous space hierarchy theorem is cleaner, and provides a tighter bound.

Intuitive Definition: A function $s(n)$ is space-constructible if a Turing Machine can mark off exactly $s(n)$ tape cells for its workspace without using more than $O(s(n))$ space to do the marking.

Definition 2.1. A function $s : \mathbb{N} \rightarrow \mathbb{N}$, where $s(n) = \Omega(n)$, is called **space-constructible**, if the function that maps 1^n (n in unary form) to the binary representation of $s(n)$ is computable in space $O(s(n))$.

All commonly occurring functions that are $\Omega(n)$ are space-constructible, including the functions $n \log n$, and n^2 .

Theorem 2.2. Let $s : \mathbb{N} \rightarrow \mathbb{N}$ be a space-constructible function, then there exists a language L that is decidable in $O(s(n))$ space, but not decidable in $o(s(n))$ space.

Corollary 2.3. For any two real numbers $1 \leq \epsilon_1 < \epsilon_2$, we have $\text{SPACE}(n^{\epsilon_1}) \subsetneq \text{SPACE}(n^{\epsilon_2})$.

Corollary 2.4. $\text{PSPACE} \subsetneq \text{EXPSPACE}$.

3 The Padding Technique

The padding technique is a method used to artificially inflate the size of an input to shift a problem into a lower complexity class relative to its new size. We use this to prove structural theorems about complexity classes.

Lemma 3.1. The following are all equivalent:

1. $\text{PSPACE} = \text{PTIME}$
2. There exists $k > 1$ s.t. $\text{SPACE}(n^k) \subseteq \text{PTIME}$
3. $\text{SPACE}(n) \subseteq \text{PTIME}$

Proof. Clearly, if $\text{PSPACE} = \text{PTIME}$ then $\text{SPACE}(n^k) \subseteq \text{PSPACE} = \text{PTIME}$ for all $k \geq 1$. Also, if $\text{SPACE}(n^k) \subseteq \text{PTIME}$ for some $k \geq 1$ then $\text{SPACE}(n) \subseteq \text{SPACE}(n^k) \subseteq \text{PTIME}$. It is left to prove that if $\text{SPACE}(n) \subseteq \text{PTIME}$ then $\text{PSPACE} = \text{PTIME}$ and for this we use a padding argument.

Assume that $\text{SPACE}(n) \subseteq \text{PTIME}$ and consider a language $L \in \text{PSPACE}$. Let $f(n) = n^k$ be such that $L \in \text{DSpace}(f(n))$. Consider the language $L' = \{w\#1^m : w \in L \text{ and } m = |w|^k\}$. Let the size of the new input string $w\#1^m$ be N . Notice that $N = |w| + 1 + |w|^k$, meaning $N = O(|w|^k)$.

It is not hard to see that $L' \in \text{SPACE}(N) \subseteq \text{PTIME}$. Indeed, $f(|w|) = |w|^k$ is space-constructible, and so we can first compute $|w|^k$ as long as the space for the output does not exceed m cells (if it does we reject), and then compare

$|w|^k$ with m . If they are not equal then we reject, and otherwise, if $m = |w|^k$, then we decide whether $w \in L$ using $O(f(|w|)) = O(|w|^k) = O(N)$ space. Because we assumed $\text{SPACE}(n) \subseteq \text{PTIME}$, it follows that $L' \in \text{PTIME}$.

It then follows that $L \in \text{PTIME}$ — On input w we first compute $m = |w|^k$ in polynomial time in $|w|$ (*why is this possible?*) and then determine whether $w\#1^m \in L'$. The latter can be done in polynomial time in $|w\#1^m| = O(|w|^k)$ which is polynomial in $|w|$. Hence, $L \in \text{PTIME}$ and we're done. $\square$

Corollary 3.2. *For all $k \geq 1$ it holds that $\text{SPACE}(n^k) \neq \text{PTIME}$*

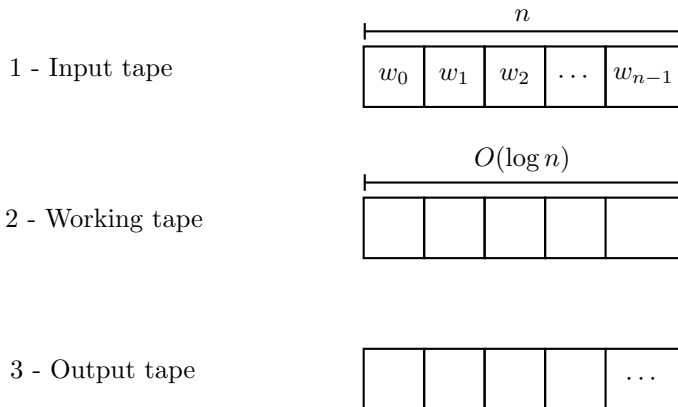
Proof. By the previous lemma, $\text{SPACE}(n^k) = \text{PTIME}$ implies $\text{PSPACE} = \text{PTIME}$, which clearly implies $\text{SPACE}(n^{k+1}) \subseteq \text{PSPACE} = \text{PTIME} = \text{SPACE}(n^k)$. The latter is a contradiction to the separation $\text{SPACE}(n^k) \subsetneq \text{SPACE}(n^{k+1})$ given by the space hierarchy theorem. $\square$

4 Logarithmic Space Languages

We would like to define logarithmic space languages. However, how can a TM given a word of size n work in $O(\log n)$ space? We solve this by altering the machine's architecture to separate the input from the workspace.

Definition 4.1 (Logarithmic Space TM). *A logarithmic space TM is a TM with 3 tapes:*

1. Input **read only** tape (contains only letters from Σ)
2. Working tape - may write any $\sigma \in \Gamma$ and utilize $O(\log n)$ space.
3. Output **write only** tape - we may write some letter $\sigma \in \Sigma$ and move right or halt on this tape.



With that we define the following languages:

$$\begin{aligned} \mathbf{L} &= \text{SPACE}(O(\log n)) \\ \mathbf{NL} &= \text{NSPACE}(O(\log n)) \end{aligned}$$

Why don't we see powers of n in the log?

Recall that $\log(n^k) = k \log n$. Thus, for every polynomial p , $\log(p(n)) = O(\log n)$. Conversely, $\log(n) = o(\log^2(n))$. So a machine that requires $O(\log^2(n))$ space does not belong to $\mathbf{L}$.

Relating $\mathbf{L}$ to $\mathbf{P}$ via Configurations:

We have seen that $\text{SPACE}(f(n)) \subseteq \text{TIME}(2^{O(f(n))})$. Thus, if $f = O(\log n)$, the maximum time an $O(\log n)$ space machine can run without infinite looping (the number of its configurations) is $n \cdot 2^{O(f(n))} = n \cdot 2^{O(\log n)} = n \cdot \text{poly}(n)$. We conclude that $\mathbf{L} \subseteq \mathbf{P}$. As shown in class, it also holds that $\mathbf{NL} \subseteq \mathbf{P}$.

5 NL-Completeness

Similarly to the question of whether $\mathbf{P} = \mathbf{NP}$, we do not know whether $\mathbf{L} = \mathbf{NL}$. As we did in $\mathbf{NP}$, we want to identify the “hardest” problems in $\mathbf{NL}$. Thus, we want to define the notion of $\mathbf{NL}$ -hardness using log-space reductions.

5.1 Log Space Reductions

Definition 5.1. We say that a language A is **log-space reducible** to B if there exists a logarithmic space computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that $\forall w, w \in A \iff f(w) \in B$. In this case we write $A \leq_L B$.

Definition 5.2. We say a language $\hat{L}$ is **NL-hard** if $\forall L' \in \text{NL}, L' \leq_L \hat{L}$.
If in addition $\hat{L} \in \text{NL}$ then we say that $\hat{L}$ is **NL-complete**.

Theorem 5.3 (Reduction Theorem for LOG-SPACE). If $B \in \text{L}$, and $A \leq_L B$, then $A \in \text{L}$.
If $B \in \text{NL}$, and $A \leq_L B$, then $A \in \text{NL}$.
Similarly, if $A \notin \text{L}$ and $A \leq_L B$ then $B \notin \text{L}$.

Corollary 5.4. If A is NL-complete and $A \in \text{L}$ then $\text{L} = \text{NL}$.

Lemma 5.5 (Transitivity). If $A \leq_L B$ and $B \leq_L C$ then $A \leq_L C$.

The proof is left as an exercise (it is similar to the proof of standard mapping reductions, requiring careful management of log-space pointers).

5.2 Relations Between Classes

So far we've seen the following relations between classes:

$$\text{L} \subseteq \text{NL} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXP}$$

From the hierarchy theorems we get the strict separations:

$$\text{L} \subsetneq \text{PSPACE}, \text{P} \subsetneq \text{EXP}$$

And a non-deterministic version of the space hierarchy theorem shows that:

$$\text{NL} \subsetneq \text{NPSPACE} = \text{PSPACE}$$

6 STRONGLY-CONNECTED is NL-Complete

A directed graph is strongly connected if every two nodes are connected by a directed path in each direction. We define the language:

$$\text{STRONGLY-CONNECTED} = \{\langle G \rangle : G \text{ is a strongly connected graph}\}$$

and we claim that it is NL-complete.

Part 1: Showing $\text{STRONGLY-CONNECTED} \in \text{NL}$

We use the Immerman-Szelepcsényi theorem:

Theorem 6.1 (Immerman). $\text{NL} = \text{coNL}$

You proved in the lecture that $\text{PATH} = \{\langle G, s, t \rangle : G \text{ is a directed graph and there exists a path from } s \text{ to } t\}$ is NL-complete. As such, its complement, $\overline{\text{PATH}}$, is also NL-complete. So, $\overline{\text{PATH}} \in \text{NL}$. Thus, there exists an NTM that given $\langle G, s, t \rangle$ accepts iff there is no path from s to t in G .

Consider the following machine which recognizes $\overline{\text{STRONGLY-CONNECTED}}$:

Given an input $\langle G \rangle$:

1. Non-deterministically select two nodes a and b .
2. Simulate the NTM that decides $\overline{\text{PATH}}$ on input $\langle G, a, b \rangle$. If it accepts, then the graph is not strongly connected so accept. Otherwise, reject.

Since storing the node pointers a and b only takes $O(\log n)$ space, and $\overline{\text{PATH}}$ is recognized in log space, we get that $\overline{\text{STRONGLY-CONNECTED}} \in \text{NL}$.

By Immerman's theorem, we get that $\text{STRONGLY-CONNECTED} \in \text{NL}$.

Part 2: Showing Hardness (Reduction from PATH)

Next we show that every other language in NL is log-space reducible to STRONGLY-CONNECTED. We do this by reducing PATH (which is NL-complete) to STRONGLY-CONNECTED.

Consider the following reduction:

On input $\langle G, s, t \rangle$ where $G = (V, E)$, the reduction outputs $\langle G' \rangle$, where G' is obtained from G by adding, for every $v \in V$, the edges (v, s) and (t, v) .

We claim that the reduction is correct, and that it can be done in logspace.

Proof of Correctness:

We need to show that if there is a path from s to t in G , then G' is strongly connected. Indeed, for any two arbitrary nodes $x, y \in V$, a path can be formed by starting with (x, s) , then taking the original path from s to t , and finally taking (t, y) . Thus, G' is strongly connected.

Conversely, we need to show that if there is no path from s to t in G , then G' is not strongly connected. Indeed, we claim that t is not reachable from s in G' . A simple path from s to t in G' cannot use any of the new edges, because the only additional edges in the constructed graph go *into* s and *out of* t . Therefore, there can be no new ways of getting from s to t . Any path would also be a path in G , which we know does not exist.

Log-Space Computability:

Finally, we must verify that the reduction can indeed be performed by a log space TM. A log space TM can start copying the input G , and for every vertex it writes, it also writes the two new edges. This is done vertex-by-vertex.

So the reduction TM will do the following:

1. Copy all of G onto the output tape.
2. For each node i in G :
 - Add an edge from i to s .
 - Add an edge from t to i .

Although the output has size $O(n)$, essentially the only space necessary to perform the reduction is that used to keep track of the node number i in the **for** loop above. Storing the index i requires exactly $O(\log n)$ space. Therefore, the reduction is safely computed in log-space.

7 E_{NFA} is NL-Complete

We define the language:

$$E_{\text{NFA}} = \{ \langle A \rangle \mid A \text{ is an NFA and } L(A) = \emptyset \}$$

and we claim that it is NL-complete.

Part 1: Showing $E_{\text{NFA}} \in \text{NL}$

We first observe that it is easier to work with the complement language:

$$\overline{E_{\text{NFA}}} = \{ \langle A \rangle \mid A \text{ is an NFA and } L(A) \neq \emptyset \}$$

An NFA accepts some string iff there exists a path in its state graph from an initial state to an accepting state. Thus, a nondeterministic algorithm for $\overline{E_{\text{NFA}}}$ works as follows:

Given an input $\langle A \rangle$ where $A = (Q, \Sigma, \delta, Q_0, F)$:

1. Nondeterministically choose a start state $q_0 \in Q_0$.
2. Nondeterministically guess a sequence of states of length at most $|Q|$.
3. Verify step-by-step that each transition is consistent with δ .
4. If an accepting state $f \in F$ is reached, accept.

Since the machine only stores a constant number of states and a counter up to $|Q|$, it uses $O(\log n)$ space. Therefore,

$$\overline{E_{\text{NFA}}} \in \text{NL}$$

and by Immerman–Szelepcsényi ($\text{NL} = \text{coNL}$), we get:

$$E_{\text{NFA}} \in \text{NL}.$$

Part 2: Showing Hardness (Reduction from PATH)

We reduce from $\text{PATH} = \{\langle G, s, t \rangle : \text{there is a path from } s \text{ to } t\}$, which is NL -complete.

Given an input $\langle G, s, t \rangle$ where $G = (V, E)$, construct an NFA

$$A = (V, \Sigma, \delta, Q_0, F)$$

as follows:

- $\Sigma = \{a\}$
- $Q_0 = \{s\}$
- $F = \{t\}$
- For every $u \in V$, define

$$\delta(u, a) = \{v \mid (u, v) \in E\}$$

Correctness:

There is a path from s to t in G iff there exists a sequence of transitions in A from the start state s to the accepting state t , i.e.,

$$\langle G, s, t \rangle \in \text{PATH} \iff L(A) \neq \emptyset.$$

Thus,

$$\text{PATH} \leq_L \overline{E_{\text{NFA}}}.$$

Log-Space Computability:

The reduction only relabels vertices as states and edges as transitions. A log-space machine can output δ by scanning the adjacency list of G and writing transitions one by one, storing only the current vertex in $O(\log n)$ space.

Therefore the reduction is computable in log space.

Conclusion:

We have shown:

$$\text{PATH} \leq_L \overline{E_{\text{NFA}}}$$

so $\overline{E_{\text{NFA}}}$ is NL -hard. Since $\overline{E_{\text{NFA}}} \in \text{NL}$, it is NL -complete.

Finally, using Immerman theorem $\text{NL} = \text{coNL}$, we conclude that:

$$E_{\text{NFA}} \text{ is } \text{NL}\text{-complete}.$$

8 10-CSS is NL-Complete

A directed graph is said to have a strongly connected component (SCC) if it is a maximal set of vertices such that every pair of vertices in the set is reachable from one another.

We define the language:

$$10-CSS = \{\langle G \rangle : G \text{ is a directed graph with exactly 10 strongly connected components}\}$$

and we claim that it is NL-complete.

Part 1: Showing $10-CSS \in \text{NL}$

We describe an NL algorithm that decides whether a graph has exactly 10 strongly connected components.

Given an input graph $\langle G \rangle$ where $G = (V, E)$:

1. Set a counter $c = 0$.
2. Iterate over all vertices $v \in V$.
3. For every vertex $u < v$, check whether:

$$u \rightsquigarrow v \quad \text{and} \quad v \rightsquigarrow u$$

where $\rightsquigarrow$ denotes path in G .

4. If there is no such vertex u , increment the counter:

$$c \leftarrow c + 1$$

(meaning that v is the first vertex encountered in its strongly connected component).

5. After iterating over all vertices, accept iff:

$$c = 10$$

The algorithm is correct because every strongly connected component has exactly one first vertex in the iteration order. Therefore, the counter is incremented exactly once for every strongly connected component.

It remains to show that the algorithm uses logarithmic space. The machine only needs to store the current vertices u, v , the counter c , and the variables needed for the reachability checks. Since the number of vertices is polynomial in the input size, storing these values requires only:

$$O(\log n)$$

space.

The reachability checks are in NL, and since $\text{NL} = \text{coNL}$, we can also check the absence of reachability in logarithmic space. Therefore, the whole algorithm works in logarithmic space, and:

$$10-CSS \in \text{NL}.$$

Part 2: Showing Hardness (Reduction from STRONGLY-CONNECTED)

We reduce from:

$$\text{STRONGLY-CONNECTED} = \{\langle G \rangle : G \text{ is strongly connected}\}$$

which is NL-complete.

Given a directed graph $G = (V, E)$, we construct a new graph G' as follows:

- Start with G .
- Add 9 new isolated vertices:

$$u_1, \dots, u_9$$

- Add no edges involving these new vertices.

Proof of Correctness:

If G is strongly connected, then G has exactly one SCC. The new vertices are isolated, so each one forms its own SCC. Therefore:

$$\#SCC(G') = 1 + 9 = 10$$

and hence:

$$G \in \text{STRONGLY-CONNECTED} \implies G' \in 10\text{-CSS}$$

Conversely, suppose that G is not strongly connected. Then G contains at least two SCCs. Adding the 9 isolated vertices gives:

$$\#SCC(G') \geq 2 + 9 = 11$$

so:

$$G' \notin 10\text{-CSS}$$

Therefore:

$$G \in \text{STRONGLY-CONNECTED} \iff G' \in 10\text{-CSS}$$

and we have a valid reduction:

$$\text{STRONGLY-CONNECTED} \leq_L 10\text{-CSS}$$

Log-Space Computability:

The reduction only adds 9 new vertices and does not modify the original graph.

A log-space machine only needs to keep track of the input vertices while copying the graph and outputting the additional vertices.

Since storing a vertex number requires $O(\log n)$ space, the reduction is computable in logarithmic space.

Conclusion:

We have shown:

$$10\text{-CSS} \in \text{NL}$$

and:

$$\text{STRONGLY-CONNECTED} \leq_L 10\text{-CSS}$$

so 10-CSS is NL-hard.

Therefore:

10-CSS is NL-complete